Page Size

1 Core Idea

Definition

Page size is the number of bytes in one virtual page and one physical page frame.

Choosing a page size is a tradeoff. Small pages reduce wasted space, but large pages reduce page-table size, disk-transfer overhead, and TLB pressure.

No Universal Best Size

There is no single optimum page size for all systems. The best choice depends on memory size, TLB size, disk behavior, process sizes, and workload.

2 Hardware and OS Choice

Hardware may support a fixed page size, such as 4096 bytes. The operating system can still treat groups of pages as larger logical pages.

Example

If hardware pages are 4 KB, the OS can treat pairs of consecutive pages as 8-KB units by allocating them together.

Modern systems may support multiple page sizes, for example small pages for user processes and large pages for the kernel or large memory regions.

3 Arguments for Small Pages

3.1 Less Internal Fragmentation

Internal Fragmentation

Internal fragmentation is wasted space inside an allocated page when the segment does not exactly fill the final page.

On average, the last page of a segment is half empty.

If:

- n = number of segments in memory;
- p = page size in bytes;

then expected wasted space is:

Internal Fragmentation

$$\boxed{\text{wasted space} \approx \frac{np}{2}}$$

Smaller p means less average waste in the final page of each segment.

3.2 Less Unneeded Program Memory Loaded

Consider a program with eight sequential phases, each 4 KB.

Page size	Memory needed at one instant
32 **KB**	Whole 32-KB region may be kept.
16 **KB**	About 16 KB may be needed.
4 **KB or smaller**	Only the current 4-KB phase may be needed.

Small-Page Benefit

Smaller pages let memory contain only the currently needed parts of a program.

4 Arguments for Large Pages

4.1 Smaller Page Tables

Small pages mean more pages per process, so page tables become larger.

Page size	Pages needed for a 32-KB program
8 **KB**	$32/8 = 4$ pages.
512 **bytes**	32 KB$/512 = 64$ pages.

More pages means more page-table entries.

4.2 Better Disk Transfer Efficiency

Disk transfers are usually done one page at a time. Much of the cost comes from seek time and rotational delay, not just the number of bytes transferred.

Case	Approximate load time
64 **pages of** 512 **bytes**	64×10 msec $= 640$ msec.
4 **pages of** 8 **KB**	4×12 msec $= 48$ msec.

Disk-Transfer Point

A small page may take almost as long to transfer as a large page, so many small transfers can be much slower.

4.3 Less TLB Pressure

TLB entries are limited. Larger pages let each TLB entry cover more memory.

Page size	TLB coverage example
4 **KB**	A 64-KB working set needs at least 16 TLB entries.
2 **MB**	A 1-MB region can fit under one large-page TLB entry in theory.

TLB Point

Large pages can improve performance because scarce TLB entries cover more address space.

5 Context Switch Cost

On some machines, the OS must load page-table information into hardware registers on each context switch.

Small-Page Cost

Smaller pages create more page-table entries, which can increase page-table loading cost during context switches.

6 Mathematical Tradeoff

Let:

- s = average process size in bytes;
- p = page size in bytes;
- e = bytes required for one page-table entry.

Approximate number of pages per process:

$$\frac{s}{p}$$

Page-table space:

$$\frac{s}{p} \times e = \frac{se}{p}$$

Average internal fragmentation in the final page:

$$\frac{p}{2}$$

Total Overhead

$$\boxed{\text{overhead}(p) = \frac{se}{p} + \frac{p}{2}}$$

7 Finding the Optimum Page Size

To minimize:

$$f(p) = \frac{se}{p} + \frac{p}{2}$$

take the derivative with respect to p:

$$f'(p) = -\frac{se}{p^2} + \frac{1}{2}$$

Set it equal to zero:

$$-\frac{se}{p^2} + \frac{1}{2} = 0$$

Move terms:

$$\frac{1}{2} = \frac{se}{p^2}$$

Multiply by $2p^2$:

$$p^2 = 2se$$

Take square root:

Optimum Page Size

$$\boxed{p = \sqrt{2se}}$$

8 Worked Example

Problem

Find the approximate optimum page size when:

$$s = 1 \text{ MB}, \qquad e = 8 \text{ bytes}$$

Solution

Use:

$$p = \sqrt{2se}$$

Assume:

$$1 \text{ MB} = 2^{20} \text{ bytes}$$

Substitute:

$$p = \sqrt{2 \times 2^{20} \times 8}$$

Since $8 = 2^3$:

$$p = \sqrt{2^1 \times 2^{20} \times 2^3}$$

$$p = \sqrt{2^{24}}$$

$$p = 2^{12} = 4096 \text{ bytes}$$

Answer

The optimum page size is approximately:

$$\boxed{4 \text{ KB}}$$

9 Summary of Tradeoffs

Page size	Advantages	Disadvantages
Small pages	Less internal fragmentation; less unused program data in RAM.	Larger page tables; more TLB entries needed; more page transfers.
Large pages	Smaller page tables; better disk transfer efficiency; better TLB coverage.	More internal fragmentation; more unneeded memory may be loaded.

10 Final Summary

One-Box Summary

Small pages reduce internal fragmentation.
Large pages reduce page-table size, disk overhead, and TLB pressure.
The simplified overhead model is $\frac{se}{p} + \frac{p}{2}$.
Its optimum is $p = \sqrt{2se}$.
For $s = 1$ MB and $e = 8$, the result is 4 KB.